

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregates for customer spending analysis.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Aggregation of customer spending metrics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Scheduling or orchestration mechanisms
- Data archival or retention policies
- User interface or reporting layer

Assumptions

1. CSV files in the source S3 paths are well-formed and use standard delimiters (comma-separated)
2. The term "Null" (string) and NULL (actual null values) both represent missing data
3. Duplicate detection is based on all columns in each dataset
4. CustId serves as the join key between customer and order datasets
5. "Customer updates" refers to any change in customer attributes (Name, EmailId, Region)
6. Date fields in Order dataset are in a parseable format (specific format not provided in BRD)

7. OpTs (Operation Timestamp) will be system-generated at processing time
8. AWS Glue Data Catalog database "gen_ai_poc_databrickscoe" already exists
9. S3 bucket paths are accessible and have appropriate permissions configured
10. Hudi table format is used exclusively for SCD2 implementation
11. TotalAmount in aggregation is calculated as $\text{PricePerUnit} \times \text{Qty}$ summed per customer
12. Aggregation table is a full refresh snapshot (not incremental)
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data from the specified S3 location and load it into a processable format for downstream transformations.

- Business Objective:

Enable access to customer master data for enrichment and tracking of customer-related transactions.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId: (datatype not specified in BRD)
 - Name: (datatype not specified in BRD)
 - EmailId: (datatype not specified in BRD)
 - Region: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
 1. Read all CSV files from the source S3 path
 2. Parse CSV content using standard comma delimiter
 3. Map columns to the defined schema: CustId, Name, EmailId, Region
 4. Load data into memory/Spark DataFrame for processing
- Outputs:
 - In-memory customer dataset ready for cleaning
- Preconditions:

- S3 path s3://adif-sdlc/sdlc_wizard/customerdata/ exists and contains at least one CSV file
- AWS Glue job has read permissions on the source path
- Postconditions:
- Customer data is loaded and available for data quality processing
- Error / Validation Conditions:
- If source path does not exist, job must fail with clear error message
- If CSV files are malformed or cannot be parsed, job must fail with file-level error details
- If schema columns are missing, job must fail with schema mismatch error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data from the specified S3 location and load it into a processable format for downstream transformations.

- Business Objective:

Enable access to order transaction data for customer spending analysis and historical tracking.

- Inputs:
- Source: s3://adif-sdlc/sdlc_wizard/orderdata/
- Format: CSV
- Schema:
- OrderId: (datatype not specified in BRD)
- ItemName: (datatype not specified in BRD)
- PricePerUnit: (datatype not specified in BRD)
- Qty: (datatype not specified in BRD)
- Date: (datatype not specified in BRD)
- CustId: (datatype not specified in BRD)
- Processing Rules / Functional Steps:
- 1. Read all CSV files from the source S3 path
- 2. Parse CSV content using standard comma delimiter
- 3. Map columns to the defined schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- 4. Load data into memory/Spark DataFrame for processing

- Outputs:
- In-memory order dataset ready for cleaning
- Preconditions:
- S3 path s3://adif-sdlc/sdlc_wizard/orderdata/ exists and contains at least one CSV file
- AWS Glue job has read permissions on the source path
- Postconditions:
- Order data is loaded and available for data quality processing
- Error / Validation Conditions:
- If source path does not exist, job must fail with clear error message
- If CSV files are malformed or cannot be parsed, job must fail with file-level error details
- If schema columns are missing, job must fail with schema mismatch error
-

FR-DQ-001: Clean Customer and Order Data

- Title: Remove Null Values and Duplicates
- Description:

The system shall apply data quality rules to both customer and order datasets by removing null values and duplicate records.

- Business Objective:

Ensure data quality and consistency by eliminating incomplete and redundant records before downstream processing.

- Inputs:
- In-memory customer dataset from FR-INGEST-001
- In-memory order dataset from FR-INGEST-002
- Processing Rules / Functional Steps:
 1. For customer dataset:
 - Remove rows where any column contains the string "Null" (case-sensitive)
 - Remove rows where any column contains NULL (actual null value)
 - Remove duplicate rows based on all columns
 2. For order dataset:
 - Remove rows where any column contains the string "Null" (case-sensitive)
 - Remove rows where any column contains NULL (actual null value)

- Remove duplicate rows based on all columns
3. Retain only unique, complete records in each dataset

- Outputs:
 - Cleaned customer dataset
 - Cleaned order dataset
- Preconditions:
 - Customer and order datasets are loaded in memory
- Postconditions:
 - Both datasets contain no null values or duplicates
 - Datasets are ready for catalog registration and transformation
- Error / Validation Conditions:
 - If all records are removed due to null values, log warning but continue processing
 - If no duplicates are found, continue processing without error
-

FR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Title: Create Glue Catalog Tables for Customer and Order
- Description:

The system shall register the cleaned customer and order datasets as tables in the AWS Glue Data Catalog for query access and metadata management.

- Business Objective:

Enable SQL-based querying and metadata-driven processing by cataloging cleaned datasets.

- Inputs:
 - Cleaned customer dataset from FR-DQ-001
 - Cleaned order dataset from FR-DQ-001
- Processing Rules / Functional Steps:
 1. Write cleaned customer dataset to Glue Data Catalog
 - Database: gen_ai_poc_databrickscoe
 - Table name: sdlc_wizard_customer
 2. Write cleaned order dataset to Glue Data Catalog
 - Database: gen_ai_poc_databrickscoe
 - Table name: sdlc_wizard_order

3. Ensure table schemas match the source CSV schemas
4. Update catalog metadata with current processing timestamp

- Outputs:

- Glue Catalog table: gen_ai_poc_databrickscoe.sdmc_wizard_customer
- Glue Catalog table: gen_ai_poc_databrickscoe.sdmc_wizard_order

- Preconditions:

- Glue Data Catalog database "gen_ai_poc_databrickscoe" exists
- AWS Glue job has permissions to create/update tables in the catalog

- Postconditions:

- Both tables are queryable via AWS Athena
- Table metadata is current and reflects the cleaned datasets

- Error / Validation Conditions:

- If database does not exist, job must fail with clear error message
- If table creation fails due to permissions, job must fail with IAM-related error
- If schema inference fails, job must fail with schema error details

-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with Historical Tracking

- Description:

The system shall join customer and order data, then implement Slowly Changing Dimension Type 2 logic to track historical changes in the ordersummary table using Apache Hudi format.

- Business Objective:

Maintain a complete historical record of order summaries that reflects changes in customer attributes over time, enabling point-in-time analysis.

- Inputs:

- Cleaned customer dataset from gen_ai_poc_databrickscoe.sdmc_wizard_customer
- Cleaned order dataset from gen_ai_poc_databrickscoe.sdmc_wizard_order
- Existing ordersummary Hudi table (if present) from s3://adif-sdmc/curated/sdmc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Join customer and order datasets on CustId

2. For each joined record, compare with existing ordersummary table:

- If no matching record exists (based on business key), insert new row with:
- IsActive = True
- StartDate = current processing date
- EndDate = NULL
- OpTs = current timestamp
- If matching record exists and customer attributes have changed:
- Update existing active row: set IsActive = False, EndDate = current processing date
- Insert new row with updated attributes: IsActive = True, StartDate = current processing date, EndDate = NULL, OpTs = current timestamp
- If matching record exists and no changes detected, no action required

3. Write results to Hudi table format at target S3 location

4. Register/update ordersummary table in Glue Data Catalog

- Outputs:
- Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
- Table includes columns: all customer fields, all order fields, IsActive, StartDate, EndDate, OpTs
- Preconditions:
- Customer and order tables exist in Glue Data Catalog
- Target S3 path is accessible with write permissions
- Hudi libraries are available in Glue job environment
- Postconditions:
- ordersummary table contains current and historical records
- All active records have IsActive = True and EndDate = NULL
- All closed records have IsActive = False and valid EndDate
- Table is queryable via Athena with time-travel capability
- Error / Validation Conditions:
- If join produces no results, log warning and create empty Hudi table
- If CustId is missing in either dataset, exclude those records and log warning
- If Hudi write fails, job must fail with Hudi-specific error details
- If duplicate business keys are detected in source, job must fail with data quality error
-

FR-INCR-001: Process Incremental Customer Updates

- Title: Trigger SCD2 Updates Based on Customer Changes

- Description:

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the ordersummary table to maintain historical accuracy.

- Business Objective:

Ensure that changes in customer master data are reflected in the order summary history, enabling accurate customer attribution over time.

- Inputs:

- New/updated customer records from s3://adif-sdlc/sdlc_wizard/customerdata/
- Existing ordersummary Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Processing Rules / Functional Steps:

1. Identify customer records that have changed since last processing run

2. For each changed customer:

- Retrieve all related active records from ordersummary table (where CustId matches and IsActive = True)

- Close existing active records: set IsActive = False, EndDate = current processing date

- Insert new active records with updated customer attributes: IsActive = True, StartDate = current processing date, EndDate = NULL, OpTs = current timestamp

3. Maintain all SCD2 tracking fields: IsActive, StartDate, EndDate, OpTs

4. Write updates to Hudi table using upsert operation

- Outputs:

- Updated ordersummary Hudi table with new historical versions

- Preconditions:

- ordersummary table exists and contains data

- Customer data source contains change indicators or full refresh data

- Postconditions:

- All customer changes are reflected in ordersummary history

- Previous versions are properly closed with EndDate populated

- New versions are active with current customer attributes

- Error / Validation Conditions:

- If no customer changes detected, skip processing and log informational message

- If customer record is deleted (not present in new data), existing active records must be closed

- If Hudi upsert fails, job must fail with transaction error details
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Spending Aggregation
- Description:

The system shall aggregate order data by customer to calculate total spending amounts and store results in a dedicated analytics table.

- Business Objective:

Provide a summarized view of customer spending for analytical queries and reporting purposes.

- Inputs:
 - Cleaned order dataset from gen_ai_poc_databrickscoe.sdmc_wizard_order
 - Cleaned customer dataset from gen_ai_poc_databrickscoe.sdmc_wizard_customer
- Processing Rules / Functional Steps:
 1. Join order and customer datasets on CustId
 2. For each customer, calculate TotalAmount = SUM(PricePerUnit × Qty)
 3. Group results by customer Name and Date
 4. Select final columns: Name, TotalAmount, Date
 5. Write aggregated results to target S3 location
 6. Register/update customeragggregatespend table in Glue Data Catalog
- Outputs:
 - Analytics table at s3://adif-sdmc/analytics/customeragggregatespend/
 - Glue Catalog table: gen_ai_poc_databrickscoe.customeragggregatespend
 - Schema: Name, TotalAmount, Date
- Preconditions:
 - Customer and order tables exist in Glue Data Catalog
 - Target S3 path is accessible with write permissions
- Postconditions:
 - customeragggregatespend table contains one row per customer per date with total spending
 - Table is queryable via Athena for analytical purposes
- Error / Validation Conditions:
 - If PricePerUnit or Qty are non-numeric, exclude those records and log warning

- If join produces no results, create empty table and log warning
- If aggregation fails due to data type issues, job must fail with calculation error details
- If multiple customers have the same Name, aggregate separately (Name is not assumed to be unique)
-

3. Non-Functional Considerations

Performance

- Processing should leverage AWS Glue's distributed Spark engine for scalability
- Hudi table format enables efficient upsert operations for SCD2 logic

Monitoring

- AWS CloudWatch should be used for job execution monitoring and logging
- Job metrics (records processed, failures, duration) should be captured

Data Governance

- AWS Lake Formation may optionally be used for fine-grained access control
- Glue Data Catalog provides centralized metadata management

Query Access

- All catalog tables should be queryable via Amazon Athena
- Hudi tables support time-travel queries for historical analysis
-

4. Requirement Traceability Notes

Requirement ID	BRD Section	Key Traceability Points	
-----	-----	-----	
FR-INGEST-001	Source Input (AWS S3), Schemas	Customer CSV source path, schema definition	
FR-INGEST-002	Source Input (AWS S3), Schemas	Order CSV source path, schema definition	
FR-DQ-001	Data Cleaning	Null removal, duplicate removal	
FR-CATALOG-001	Glue Catalog Tables	Database and table names	
FR-SCD2-001	SCD Type 2 (Hudi on S3)	Target table, location, SCD2 logic, tracking fields	
FR-INCR-001	Incremental Logic	Customer update triggers, SCD2 maintenance	
FR-AGG-001	Aggregation Table	Target table, location, aggregation logic	

- Cross-Requirement Dependencies:
- FR-DQ-001 depends on FR-INGEST-001 and FR-INGEST-002
- FR-CATALOG-001 depends on FR-DQ-001

- FR-SCD2-001 depends on FR-CATALOG-001
- FR-INCR-001 depends on FR-SCD2-001
- FR-AGG-001 depends on FR-CATALOG-001
- End of Functional Requirements Document